

A Computational Approach on Faulhaber's Polynomials and Their Divisibility

Doruk Üstündağ

Izmir Institute of Technology

23.05.2025



- 1 Preliminaries
- 2 Historical Overview
- 3 Matrix Form in Computation
- 4 Binomial Basis in Computation
- 5 Why did we rely on "Binomial Basis" for divisibility?
- 6 References

Power Sums as Faulhaber's Formula

$$\begin{aligned} S_k(n) &= \sum_{i=1}^n i^k = 1^k + 2^k + 3^k + \cdots + (n-1)^k + n^k \\ &= \frac{1}{k+1} \sum_{i=0}^k \binom{k+1}{i} B_i n^{k-i+1} \end{aligned}$$

A Brief History on Computation

A. W. F. Edwards

- Sums of powers of integers: a little of the history (Edwards,1982)
- Memory of Faulhaber
- Matrix Form
- Pascal Triangle

Donald E. Knuth

- Johann Faulhaber and Sums of Powers (Knuth, 1992)
- Calculations of Faulhaber
- More computational(!) approach

Matrix Form of Faulhaber's Formula

First five power sums

$$\sum_{k=1}^n k^0 = 1n$$

$$\sum_{k=1}^n k^1 = \frac{1}{2}n + \frac{1}{2}n^2$$

$$\sum_{k=1}^n k^2 = \frac{1}{6}n + \frac{1}{2}n^2 + \frac{1}{3}n^3$$

$$\sum_{k=1}^n k^3 = 0n + \frac{1}{4}n^2 + \frac{1}{2}n^3 + \frac{1}{4}n^4$$

$$\sum_{k=1}^n k^4 = -\frac{1}{30}n + 0n^2 + \frac{1}{3}n^3 + \frac{1}{2}n^4 + \frac{1}{5}n^5$$

Matrix Form of Faulhaber's Formula

Interpreting polynomials as products of matrices

$$\begin{pmatrix} \sum k^0 \\ \sum k^1 \\ \sum k^2 \\ \sum k^3 \\ \sum k^4 \end{pmatrix} = G_5 \begin{pmatrix} n \\ n^2 \\ n^3 \\ n^4 \\ n^5 \end{pmatrix}$$

G_5

$$G_5 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{6} & \frac{1}{2} & \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{4} & \frac{1}{2} & \frac{1}{4} & 0 \\ -\frac{1}{30} & 0 & \frac{1}{3} & \frac{1}{2} & \frac{1}{5} \end{pmatrix}$$

Matrix Form of Faulhaber's Formula

$$G_5^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ -1 & 2 & 0 & 0 & 0 \\ 1 & -3 & 3 & 0 & 0 \\ -1 & 4 & -6 & 4 & 0 \\ 1 & -5 & 10 & -10 & 5 \end{pmatrix} = \bar{A}_5$$

seems more familiar than the G_5 matrix.

Getting closer to Pascal Matrix

Let A_5 be the matrix which is obtained by changing $a_{i,j}$ of $\bar{A}_5$ into $(-1)^{i+j}a_{i,j}$.

$$A_5 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \\ 1 & 3 & 3 & 0 & 0 \\ 1 & 4 & 6 & 4 & 0 \\ 1 & 5 & 10 & 10 & 5 \end{pmatrix}$$

Matrix Form of Faulhaber's Formula

Converse of the Idea

Let $\bar{G}_5$ be the matrix which can be obtained by changing $g_{i,j}$ of G_5 into $(-1)^{i+j}g_{i,j}$. It is trivial to see that:

$$A_5^{-1} = \bar{G}_5$$

At general

For each positive integer m , $G_m^{-1} = \bar{A}_m$ and $\bar{G}_m^{-1} = A_m$

A Different Linear System for Faulhaber's Polynomials

Theorem (Wang et al., 2014)

(a) $S_p(n)$ is a $(p + 1)^{th}$ degree polynomial function of n :

$$S_p(n) = \sum_{i=0}^{p+1} a_i n^i.$$

(b) Sum of all polynomial coefficients equals to 1:

$$\sum_{i=0}^p a_i = 1.$$

(c) Constant term of these polynomials are zero:

$$a_0 = 0.$$

A Different Linear System for Faulhaber's Polynomials

Some Sketch of Idea

By previous theorem we already know that $S_p(n) = \sum_{i=0}^{p+1} a_i n^i$. To find the coefficient a_i we can construct a special linear system with $n = 1, \dots, p+1$ such that:

$$a_{p+1}1^{p+1} + a_p1^p + \dots + a_11 = S_p(1)$$

$$a_{p+1}2^{p+1} + a_p2^p + \dots + a_12 = S_p(2)$$

...

$$a_{p+1}p^{p+1} + a_pp^p + \dots + a_1p = S_p(p)$$

$$\begin{aligned} a_{p+1}(p+1)^{p+1} + a_p(p+1)^p + \dots \\ + a_1(p+1) = S_p(p+1) \end{aligned}$$

A Different Linear System for Faulhaber's Polynomials

In Matrix Form

$$A = \begin{pmatrix} 1^{p+1} & 1^p & \dots & 1^2 & 1 \\ 2^{p+1} & 2^p & \dots & 2^2 & 2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ p^{p+1} & p^p & \dots & p^2 & p \\ (p+1)^{p+1} & (p+1)^p & \dots & (p+1)^2 & (p+1) \end{pmatrix}$$

$$x = \begin{pmatrix} a_{p+1} \\ a_p \\ \vdots \\ a_2 \\ a_1 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} S_p(1) \\ S_p(2) \\ \vdots \\ S_p(p) \\ S_p(p+1) \end{pmatrix}$$

A Different Linear System for Faulhaber's Polynomials

Sketch Proof and New Algorithm

It can be trivially shown that coefficient matrix A has linearly independent rows and columns. So, we can say that:

$$Ax = B$$

has a unique solution. Whole algorithm depends on this fact. With using "Backwards Substitution" and "Gaussian Elimination" methods we can show the proposed algorithm

Algorithm for Generation of Coefficients of Faulhaber's Polynomials

ALGORITHM:

- Input** (a) the positive power p ;
(b) $a_{ij} = i^{p+2-j}$, $i, j = 1, \dots, p+1$;
(c) $a_{i(p+2)} = \sum_{k=1}^i k^p$, $i = 1, \dots, p+1$.
- Output** Coefficients of the polynomial $a_{p+1}, a_p, \dots, a_2, a_1$.
- Step 1** For $i = 1, \dots, p$, do step 2 (*Elimination Process*.)
- Step 2** For $j = i+1, \dots, p+1$ do Steps 3 and 4
- Step 3** Set $m_{ji} = \frac{a_{ji}}{a_{ii}}$.
- Step 4** Perform Row Reduction $(E_j - m_{ji}E_i) \rightarrow (E_j)$.
- Step 5** Set $a_1 = \frac{a_{(p+1)(p+2)}}{a_{(p+1)(p+1)}}$. (*Start backward substitution*.)
- Step 6** For $i = p, \dots, 1$ set $a_{p+2-i} = \frac{a_{i(p+2)} - \sum_{j=i+1}^{p+1} a_{ij} a_{p+2-j}}{a_{ii}}$.
- Step 7** Output $(a_{p+1}, a_p, \dots, a_1)$;
STOP.

Figure: (Wang et al., 2014)

SageMath Implementation of Algorithm

Implementation

Algorithm proposed by Wang et al. (2014) was firstly written in Maple.

In SageMath

```
def Faulhabers_Polynomial(p):  
    A = matrix([[i^(p + 2 - j) for j in range(1, p + 2)] for i in range  
                (1, p + 2)])  
    b = matrix(p + 1, 1, lambda i, j: sum(k^p for k in range(1, i + 2)))  
    A_augmented = A.augment(b, subdivide=True)  
    X= A_augmented.rref()  
    X_augmented = X[:, -1]  
    augmented_vector = vector(X_augmented)  
    R.<x> = PolynomialRing(QQ)  
    polynomial = sum(n * x^i for i, n in enumerate(augmented_vector  
        [::-1], start=1))  
    return (polynomial)
```

Binomial Basis Approach on Faulhaber's Polynomials

Motivation

- Optimized way for computing
- More combinatorial and algebraic approach
- No need for Bernoulli Numbers or any kind of special numbers
- Sagemath support

Integer-valued Polynomial Rings

The integer-valued polynomial ring over a base ring $\mathbf{R}$. Integer-valued polynomial rings are commutative and associative algebras, with a basis indexed by nonnegative integers. There are two natural bases, made of the sequence $\binom{x}{n}$ for $n \geq 0$ (the binomial basis) and of the other sequence $\binom{x+n}{n}$ for $n \geq 0$ (the shifted basis). This package is found built-in in Sagemath by Frederic Chapoton.

Binomial Basis Approach on Faulhaber's Polynomials

Definition (Binomial Basis)

We define the rational polynomials (b_n) for $n \in \mathbb{N}$ as follows:

$$b_0 = 1,$$

$$b_n(x) = \frac{1}{n!} \prod_{i=0}^{n-1} x - i, \text{ where } n > 0.$$

Each b_i is a rational polynomial of degree i . We call this the *binomial basis* on $\mathbb{Q}[x]$. Name originated as such because for any natural numbers n and k , one has

$$\binom{n}{k} = b_k(n).$$

Theorems for Binomial Basis

Lemma (Göksu, Ü.)

Suppose A and B are rational polynomials. If the polynomial division of A by B has nonzero remainder, then $B \nmid A$.

Theorem (Göksu, Ü.)

The binomial basis is indeed a basis for $\mathbb{Q}[x]$. Moreover, a rational polynomial is integer valued if and only if its coefficients in the binomial basis are integers.

Formula for Faulhaber's Polynomial in Binomial Basis

$$S_n(x-1) = \sum_{i=1}^{n+1} \left\{ \begin{matrix} n \\ i-1 \end{matrix} \right\} (i-1)! b_i(x)$$

Stirling Numbers

```
stirling_second_cache = {}

def stirling_second_kind(n, k):
    if (n, k) in stirling_second_cache:
        return stirling_second_cache[(n, k)]

    if n == 0 and k == 0:
        result = 1
    elif n == 0 or k == 0 or k > n:
        result = 0
    elif k == n:
        result = 1
    else:
        result = k * stirling_second_kind(n - 1, k) +
            stirling_second_kind(n - 1, k - 1)

    stirling_second_cache[(n, k)] = result
    return result
```

SageMath Implementation of Binomial Basis

Generating Faulhaber's in Binomial Basis

```
def sn(n):  
    result = [stirling_second_kind(n, k) * factorial(k) for k in range  
              (0, n+1)]  
    result.insert(0, 0)  
    polynomial = sum(y * B[i] for i, y in enumerate(result))  
    return (polynomial)
```

Degree of a Polynomial

```
def deg(a):  
    d= -1  
    while a!=0*B[0]:  
        a= a.variable_shift()-a  
        d+= 1  
    return d
```

Polynomial Long Division

```
def poly_division(c,a):  
    m= (deg(c))  
    n= (deg(a))  
    q= 0* B[0]  
    while m >= n:  
        S= (c.leading_coefficient() / (a.leading_coefficient()* binomial(  
            m,n))) * B[m-n]  
        q+=S  
        c-= S*a  
        m= deg(c)  
    return (q,c)
```

Computing in Binomial Basis

Running Time

Polynomials	Matrix Form ($O(n^3)$)	Binomial Basis Form ($O(n^2)$)
$S_1(x)$	0.0008 sec	0.0003 sec
$S_3(x)$	0.0010 sec	0.0007 sec
$S_{500}(x)$	115 sec	0.7 sec
$S_{1000}(x)$	? sec	3 sec

Better understanding of checking divisibility

By using the theorem and checking the quotient we can trivially see integer valued polynomials and the division integrally.

- Edwards, A. W. F. (1982). Sums of powers of integers: A little of the history. *The Mathematical Gazette*, 66(435), 22.
<https://doi.org/10.2307/3617302>
- Göksu, D., & Üstündağ, D. (2025). Works on Faulhaber's polynomials and their divisibility properties [Preprint].
- Knuth, D. E. (1993). Johann Faulhaber and sums of powers. *Mathematics of Computation*, 61(203), 277–294.
<https://doi.org/10.1090/S0025-5718-1993-1197512-7>
- Wang, M. J., Goel, S., & Mao, G. (2014). A new algorithm to generate a formula for the sum of integer powers. In *Proceedings of the 2014 ACM Southeast Conference (ACMSE '14)* (Article 50, pp. 1–4). Association for Computing Machinery.
<https://doi.org/10.1145/2638404.2638525>